

Bank of Questions Data Structures 2021

Data Structure Multiple Choice Questions (MCQs):

1. Process of inserting an element in stack is called _____
 - a. Create
 - b. Push
 - c. Evaluation
 - d. Pop
2. Process of removing an element from stack is called _____
 - a. Create
 - b. Push
 - c. Evaluation
 - d. Pop
3. In a stack, if a user tries to remove an element from empty stack it is___
 - a. Underflow
 - b. Empty collection
 - c. Overflow
 - d. Garbage Collection
4. Pushing an element into stack already having five elements and stack size of 5, then stack becomes
 - a. Overflow
 - b. Crash
 - c. Underflow
 - d. User flow
5. Which of the following not applications may use a stack?
 - a. A parentheses balancing program
 - b. Tracking of local variables at run time
 - c. Compiler Syntax Analyzer
 - d. Data Transfer between two asynchronous process

6. What is the value of the postfix expression 6 3 2 4 + - *:
- a. 1
 - b. 40
 - c. 74
 - d. -18
7. The postfix form of the expression $(A + B) * (C * D - E) * F / G$ is?
- a. $AB + CD * E - FG /**$
 - b. $AB + CD * E - F **G /$
 - c. $AB + CD * E - *F *G /$
 - d. $AB + CDE * - * F *G /$
8. The data structure required to check whether an expression contains balanced parenthesis is?
- a. Stack
 - b. Queue
 - c. Array
 - d. Tree
9. What data structure would you mostly likely see in a non-recursive implementation of a recursive algorithm?
- a. Linked List
 - b. Stack
 - c. Queue
 - d. Tree
10. The postfix form of $A * B + C / D$ is?
- a. $*AB / CD +$
 - b. $AB * CD / +$
 - c. $A * BC + / D$
 - d. $ABCD + / *$

11. Which data structure is needed to convert infix notation to postfix notation?
- a. Branch
 - b. Tree
 - c. Queue
 - d. Stack
12. The prefix form of $A-B / (C * D ^ E)$ is?
- a. $-/*^ACBDE$
 - b. $-ABCD*^DE$
 - c. $-A/B*C^DE$
 - d. $-A/BC*^DE$
13. What is the result of the following operation? Top (Push (S, X))
- a. X
 - b. X+S
 - c. S
 - d. XS
14. The prefix form of an infix expression $(p + q) - (r * t)$ is?
- a. $+ p q - *rt$
 - b. $- +p q r * t$
 - c. $- +p q * rt$
 - d. $- + * p q r t$
15. Which data structure is used for implementing recursion?
- a. Queue
 - b. Stack
 - c. Array
 - d. List

16. Convert the following infix expressions into its equivalent postfix expressions $(A+B\wedge D)/(E-F) +G$
- a. $(A\ B\ D\ \wedge\ +\ E\ F\ -\ /\ G\ +)$
 - b. $(ABD\ +\ \wedge\ E\ F\ -\ /\ G\ +)$
 - c. $(A\ B\ D\ \wedge\ +\ E\ F\ /\ -\ G\ +)$
 - d. $(A\ B\ D\ E\ F\ +\ \wedge\ /\ -\ G\ +)$
17. Which of the following statement(s) about stack data structure is/are NOT correct?
- a. Linked List are used for implementing Stacks
 - b. Top of the Stack always contain the new node
 - c. Stack is the FIFO data structure
 - d. Null link is present in the last node at the bottom of the stack
18. If the elements "A", "B", "C" and "D" are placed in a stack and are deleted one at a time, what is the order of removal?
- a. ABCD
 - b. DCBA
 - c. DCAB
 - d. ABDC
19. A linear list of elements in which deletion can be done from one end front and insertion can take place only at the other end rear is known as a?
- a. Queue
 - b. Stack
 - c. Tree
 - d. Linked list

20. A queue follows _____
- a. FIFO (First in First Out) principle
 - b. LIFO (Last in First Out) principle
 - c. Ordered array
 - d. Linear tree
21. If the elements "A", "B", "C" and "D" are placed in a queue and are deleted one at a time, in what order will they be removed?
- a. ABCD
 - b. DCBA
 - c. DCAB
 - d. ABDC
22. A data structure in which elements can be inserted or deleted at/from both the ends but not in the middle is?
- a. Queue
 - b. Circular queue
 - c. Dequeue
 - d. Priority queue
23. A normal queue, if implemented using an array of size MAX_SIZE, gets full when
- a. $\text{Rear} = \text{MAX_SIZE} - 1$
 - b. $\text{Front} = (\text{rear} + 1) \bmod \text{MAX_SIZE}$
 - c. $\text{Front} = \text{rear} + 1$
 - d. $\text{Rear} = \text{front}$
24. Which of the following is not the type of queue?
- a. Ordinary queue
 - b. Single ended queue
 - c. Circular queue
 - d. Priority queue

25. A linear collection of data elements where the linear node is given by means of pointer is called?
- a. **Linked list**
 - b. Node list
 - c. Primitive list
 - d. Unordered list
26. In linked list each node contains minimum of two fields. One field is data field to store the data second field is?
- a. Pointer to character
 - b. Pointer to integer
 - c. **Pointer to node**
 - d. Node
27. Which of the following properties is associated with a queue?
- a. First in Last Out
 - b. **First in First Out**
 - c. Last in First Out
 - d. Last in Last Out
28. In linked list implementation of a queue, where does a new element be inserted?
- a. At the head of link list
 - b. At the center position in the link list
 - c. **At the tail of the link list**
 - d. At any position in the linked list
29. Identify the infix expression from the list of options given below.
- a. **$a/b+(c-d)$**
 - b. $a\ b\ c^*+d\ +a\ b\ +\ c\ d\ +\ *\ c\ e\ -\ f\ -$
 - c. $a\ b\ -\ c\ -$
 - d. $+ a\ b$

30. Which of the following statement is incorrect with respect to evaluation of infix expression algorithm?

- a. Operand is pushed on to the stack
- b. If the precedence of operator is higher, pop two operands and evaluate
- c. If the precedence of operator is lower, pop two operands and evaluate
- d. The result is pushed on to the operand stack

31. Evaluate the following statement using infix evaluation algorithm and choose the correct answer. $1+2*3-2$

- a. 3
- b. 6
- c. 5
- d. 4

32. Evaluate the following statement using infix evaluation algorithm and choose the correct answer. $4*2+3-5/5$

- a. 10
- b. 11
- c. 16
- d. 12

33. Which of the following is an example for a postfix expression?

- a. $a * b (c + d)$
- b. $a b c * + d e - +$
- c. $+ a b$
- d. $a + b - c$

34. Which of the following is not an application of stack?
- a. evaluation of postfix expression
 - b. conversion of infix to postfix expression
 - c. balancing symbols
 - d. line at ticket counter
35. What would be the Prefix notation for the given equation?
 $(A * B) + (C * D)$
- a. $+ * A B * C D$
 - b. $* + A B * C D$
 - c. $* * A B + C D$
 - d. $+ * B A * C D$
36. When an operand is read, which of the following is done?
- a. It is placed on to the output
 - b. It is placed in operator stack
 - c. It is ignored
 - d. Operator stack is emptied
37. Which of the following is an infix expression?
- a. $(a + b) * (c + d)$
 - b. $a b + c *$
 - c. $+ a b$
 - d. $a b c + *$
38. What is the postfix expression for the corresponding infix expression? $A + b * c + (d * e)$
- a. $a b c * + d e * +$
 - b. $a b c + * d e * +$
 - c. $a + b c * d e + *$
 - d. $a b c * + (d e) * +$

39. What is the corresponding postfix expression for the given infix expression? $a * (b + c) / d$

- a. $a b * + c d /$
 - b. $a b + * c d /$
 - c. $a b c * + / d$
 - d. $a b c + * d /$
-

Mark the following statements as true or false: -

1. Linked list nodes **are normally** stored contiguously in memory
(False) **are not normally**
2. A linked list is a dynamic data structure (True)
3. To traverse a linked list, the program must use a pointer different than the head pointer of the list, initialized to the first node in the list (True)
4. A stack is a data structure in which the items are added and deleted from one end only. (True)
5. A stack is a Last in First Out (LIFO) data structure (True)
6. A stack can be implemented as an array or a linked list. (True)
7. The middle elements of a stack should not be accessed directly (True)
8. Stacks are restricted versions of arrays and linked lists (True)
9. In a linked list, the order of the elements is determined by the order in which the nodes were created to store the elements. (False)
10. A single linked list can be traversed in one direction. (True)
11. A stack is referred to as a last-in, first-out (LIFO) data structure (True)
12. A stack is a **First in First Out (FIFO)** data structure.
(False) **Last in First Out LIFO**
13. A queue is referred to as a **Last Input First Output** data structure
(False) **First in First Out (FIFO)**

Fill in the blanks in each of the following:

1. The pointer to the next node in a linked list is referred to as a [link](#).
 2. A [queue](#) is a constrained version of a linked list in which nodes can be inserted only at the end of the list and deleted only from the start of the list.
 3. A stack is referred to as a [LIFO last in first out](#) data structure, because the last node inserted is the first node removed.
 4. The first node of a tree is [the root](#) node.
-

Explain the following terms

1. Data Structure:

[refers to the way a programmer views and treats a collection of data and it has everything about how that data is accessed and manipulated](#)

2. Abstract Data Type (ADT):

[It is a set of data values and associated operations that are precisely specified independent of any particular implementation.](#)

3. Different types of linked list:

- 1. Singly linked list:** begins with a pointer to the first node, and each node contains a pointer to the next node.
- 2. circular, singly linked list:** begins with a pointer to the first node, and each node contains a pointer to the next node, the "last node" does not contain a 0 pointer; rather, the pointer in the last node points back to the first node, thus closing the "circle".
- 3. Doubly linked list:** Each node has both a forward pointer to the next node in the list in the forward direction and a backward pointer to the next node in the list in the backward direction.

4. Circular, double linked list: the forward pointer of the last node points to the first node, and the backward pointer of the first node points to the last node.

4. The basic operations of Stack:

1. **initialize Stack:** Initializes the stack to an empty state.
2. **is Empty:** Determines whether the stack is empty. If the stack is empty, it returns the value true; otherwise, it returns the value false.
3. **is Full:** Determines whether the stack is full. If the stack is full, it returns the value true; otherwise, it returns the value false.
4. **push:** Adds a new element to the top of the stack.
5. **pop:** Removes the top element of the stack.
6. **top:** Returns the top element of the stack.
7. **print Stack:** print all the data in the stack.

5. The basic operations of linked lists:

1. Insert a value at the beginning of the list,
2. insert a value at the end of the list,
3. delete a value from the beginning of the list,
4. delete a value from the end of the list and
5. end the list processing.

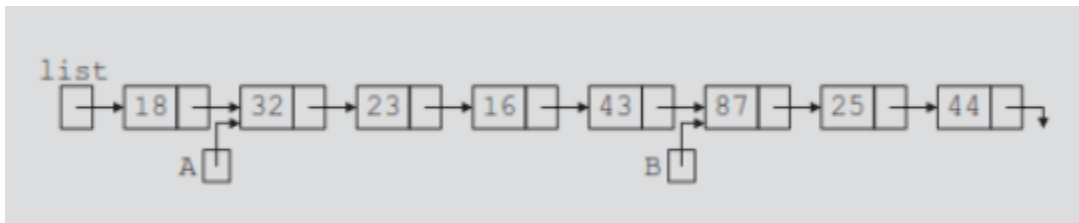
6. The basic operations of queue:

1. initialize Queue: Initializes the queue to an empty state.
2. Is Queue Empty: Determines whether the queue is empty. If the queue is empty, it returns the value true; otherwise, it returns the value false.
3. Is Queue Full: Determines whether the queue is full. If the queue is full, it returns the value true; otherwise, it returns the value false.
4. enqueue: Insert an item at the rear end of a non-full queue.
5. dequeue: Remove item from the front end of a non-empty queue.
6. print Queue: print all the data in the queue.

7. What are the differences between a stack and a queue?

stack	queue
1. A stack is an ordered group of homogeneous items (elements). 2. Stack data structure allows nodes to be added to the stack and removed from the stack only at the top. 3. For this reason, a stack is referred to as a last-in, first out (LIFO) data structure.	1. It is an ordered homogeneous group of elements in which new elements are added at one end "rear" and elements are removed from the other end "front". 2. it is a First-In-First-Out (FIFO) process where the first element in the queue will be the first one out.

Consider the linked list shown in the following figure. Assume that list, A, and B are pointers of type node.



1. Write C++ statements to do the following:

1. Make A point to the node containing 23: `A = A->link;`
2. Make list point to the node containing 16: `list = A->link->link;`
3. Make B point to the last node in the list: `B = B->link->link;`
4. Make list point to an empty list: `list = NULL;`
5. Set the value of the node containing 25 to 35: `B->link->info = 35;`

6. Create and insert the node with 10 after the node pointed to by A:

```
newNode = new nodeType;
```

```
newNode->info = 10;
```

```
newNode->link = A->link;
```

```
A->link = newNode;
```

7. Delete the node with 23. Also, deallocate the memory occupied by this node:

```
p = A->link;
```

```
A->link = p->link;
```

```
delete p;
```

8. What is the output of each of the following C++ statements?

Statements	Out put
cout << A-> info;	32
cout << list -> link -> info;	32

Write C++ statements to do the following:

1. Push element to the stack using linked list.

```
void push(int val){
    node *newNode = new node;
    newNode -> data = val;
    if(isEmpty()){
        newNode -> next = NULL;
    } else {
        newNode -> next = top;
    }
    top = newNode;
}
```

2. Write program insert an element into the queue

```
|  
void enqueue(int value){  
    if(isFull()){  
        cout << "Queue is Full";  
        return;  
    } else if(isEmpty()){  
        Front = rear = 0;  
    } else{  
        rear++;  
    }  
    arr[rear] = value;  
}
```

3. Write program delete an element from front of the queue

```
void dequeue(){  
    if(isEmpty()){  
        cout << "Queue is Empty";  
        return;  
    } else if(Front == rear){  
        Front = rear = -1;  
    } else{  
        Front++;  
    }  
}
```

4. Write program insert an element into the stack

```
void push(int val){  
    if(isFull()){  
        cout << " stack is Full.";  
        return;  
    }  
    top++;  
    arr[top] = val;  
}
```

5. Write a program to implement stack using array with all function

```
1  #include <iostream>
2  using namespace std;
3  const int MAX = 100;
4  class Stack{
5  private:
6      int arr[MAX];
7      int top;
8  public:
9      // Constructor
10     Stack(){
11         top = -1;
12     }
13     void push(int val){
14         if(isFull()){
15             cout << " stack is Full.";
16             return;
17         }
18         top++;
19         arr[top] = val;
20     }
21     void pop(){
22         if(isEmpty()){
23             cout << " stack is Empty.";
24             return;
25         }
26         top--;
27     }
28     void printAll(){
29         if(isEmpty()){
30             cout << " stack is Empty.";
31             return;
32         }
33         while(!isEmpty()){
34             cout << arr[top] << " ";
35             pop();
36         }
37     }
38     bool isEmpty(){
39         if(top == -1)
40             return 1;
41         return 0;
42     }
43     bool isFull(){
44         if(top == MAX-1)
45             return 1;
46         return 0;
47     }
48     int getTop()
49     {
50         if (isEmpty()){
51             cout << "Empty\n";
52         }
53         return arr[top];
54     }
55     // Destructor
56     ~Stack(){
57         delete arr;
58     }
59 };
60
```

6. Write program delete an element from the stack

```
void pop() {
    if (isEmpty()) {
        cout << " stack is Empty.";
        return;
    }
    top--;
}
```

7. Extend the class List by adding the function MinElement () to find the minimum element in a List.

```
int minElement() {
    if (isEmpty()) {
        cout << "Empty linkedlist!" << endl;
        return;
    }
    node *temp = head;
    int mn = 1000000000;
    while (temp != NULL) {
        mn = min(mn, temp->data);
        temp = temp->next;
    }
    return mn;
}
```

8. Extend the class List by adding the function maxElement () to find the maximum element in a List.

```
int maxElement() {
    if (isEmpty()) {
        cout << "Empty linkedlist!" << endl;
        return;
    }
    node *temp = head;
    int mx = -1;
    while (temp != NULL) {
        mx = max(mx, temp->data);
        temp = temp->next;
    }
    return mx;
}
```

9. At linked list program write function that performs Insert a value at the beginning of the list.

```
void insertFirst(int val)
{
    node* newNode = new node;
    newNode -> data = val;
    newNode -> next = head;
    head = newNode;
}
```

10. At linked list program write the function that performs Insert a value at the end of the list.

```
void insertLast(int val) {
    node* newNode = new node;
    newNode -> data = val;
    newNode -> next = NULL;

    if(head == NULL) {
        head = newNode;
    } else {

        node* temp = head;
        while(temp -> next != NULL) {
            temp = temp -> next;
        }
        temp->next = newNode;
    }
}
```

11. At linked list program write functions that Delete a value/ remove nodes from the end

```
void deleteLast()
{
    if(head==NULL) {
        cout<<"Already Empty\n";
    } else {
        node* temp = head;
        while(temp -> next -> next != NULL)
        {
            temp = temp -> next;
        }
        node *tempNext = temp -> next;
        delete tempNext;
        temp -> next = NULL;
    }
}
```

12. Push element to the stack using linked list

```
void push(int val){
    node *newNode = new node;
    newNode->data = val;
    if(isEmpty()){
        newNode->next = NULL;
    } else {
        newNode->next = top;
    }
    top = newNode;
}
```

13. Write the function that make add element in circular Queue

```
void enqueue(int val){
    if ( (rear+1)%mx == frnt){
        cout << "error";
        return;
    }else if ( isEmpty()){
        frnt = rear = 0;
    }else {
        rear = ( rear+1 )% mx;
    }
    cnt++;
    arr[rear] = val;
}
```

14. Write a C++program to read some string from the user, and then displays it again in reverse order using Stacks.

```
#include <bits/stdc++.h>
using namespace std;
stack<char> s;
int main(){
    int T;
    cout << "Enter the number of strings\n";
    cin >> T;
    while (T--){
        string str;
        cout << "Enter a string\n";
        cin >> str;
        for (int i = 0; i < str.size(); ++i) {
            s.push(str[i]);
        }
        while (!s.empty()){
            cout << s.top();
            s.pop();
        }
        s.empty();
        cout << endl;
    }
    return 0;
}
```

15. Write a function that will reverse a linked list while traversing it only once. At the conclusion, each node should point to the node that was previously its predecessor; the head should point to the node that was formerly at the end, and the node that was formerly first should have a NULL link.

```
void revers() {
    if (isEmpty() || head -> next == NULL) {
        return;
    }

    node *pre = NULL;
    node *curr = head;
    node *next = NULL;

    while (curr -> next != NULL) {
        next = curr -> next;
        curr -> next = pre;
        pre = curr;
        curr = next;
    }

    head = curr;
    head -> next = pre;
}
```

16. At circular queue program write function that performs remove from the queue.

```
void dequeue() {
    if (isEmpty()) {
        cout << "It is Empty";
        return;
    }
    if (Front == Rear) {
        Front = Rear = -1;
    } else {
        Front = (Front + 1) % MAX;
    }
}
```

17. Suppose that queue is a queueType object and the size of the array implementing queue is 100. Also, suppose that the value of queueFront is 98 and the value of queueRear is 12. What are the values of queue Front and queueRear after adding two elements to queue?

front: 98, rear: 14

18. Suppose that queue is a queueType object and the size of the array implementing queue is 100. Also, suppose that the value of queueFront is 99 and the value of queueRear is 25. What are the values of queueFront and queueRear after adding two elements and removes three elements from the queue?

front: 2, rear: 27

19. Suppose that queue is a queueType object and the size of the array implementing queue is 100. Also, suppose that the value of queueFront is 99 and the value of queueRear is 25.

A. What are the values of queueFront and queueRear after adding an element to queue?

Front: 99, Rear: 26

B. What are the values of queueFront and queueRear after removing an element from queue?

Front: 0, Rear: 25

20. Extend the class binaryTree by adding the function SumofElement to return the summation of all the elements in the binary tree.

```
int SumofElement(binaryTreeNode* t){
    if( t == NULL)
        return 0;
    return( t->info + SumofElement ( t->llink ) +
        SumofElement ( t->rlink ));
}
```

21. Extend the class bSearchTree by adding the function MinElement to find the minimum element in a bSearchTree.

```
int MinElement(binaryTreeNode* t)
{
    if ( t->llink == NULL )
        return t->info;
    else
        return ( MinElement ( t->llink ) );
}
```

22. Write code implementation of linked list that do the following:

1. insert 5 integer number at the front of the list

2. delete the number from the back of the list

3. print the results?

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  class Node{
4  public:
5      int data;
6      Node* next;
7  };
8  class LinkedList{
9  private:
10     Node* head;
11 public:
12     LinkedList(){
13         head = nullptr;
14     }
15     bool isEmpty(){
16         return head == nullptr;
17     }
18     void insertFirst(int value){
19         Node* newNode = new Node;
20         newNode->data = value;
21         newNode->next = nullptr;
22         if (isEmpty()){
23             head = newNode;
24         } else{
25             newNode->next = head;
26             head = newNode;
27         }
28     }
29     void deleteEnd(){
30         if (isEmpty()){
31             cout << "List is Empty" << endl;
32             return;
33         }
34         Node* tempNode = head;
35         while (tempNode->next->next != nullptr){
36             tempNode = tempNode->next;
37         }
38         delete tempNode->next;
39         tempNode->next = nullptr;
40     }
41     void PrintList() {
42         Node* temp = head;
43         while(temp != nullptr) {
44             cout << temp->data << " ";
45             temp = temp->next;
46         }
47     }
48 };
;
```

```

49  int main() {
50      LinkedList ll;
51      ll.insertFirst(10);
52      ll.insertFirst(20);
53      ll.insertFirst(30);
54      ll.insertFirst(40);
55      ll.insertFirst(50);
56      ll.deleteEnd();
57      ll.PrintList();
58      return 0;
59  }
60

```

23. Write the definition of the function nodeCount that returns the number of nodes in a binary tree.

```

int nodeCount(treeNode *p) {
    if (p == NULL)
        return 0;
    else
        return 1 + nodeCount(p->llink) + nodeCount(p->rlink);
}

```

24. Write the definition of the function leavesCount that takes as a parameter a pointer to the root node of a binary tree and returns the number of leaves in a binary tree.

```

int leavesCount(treeNode *p) {
    if (p == NULL)
        return 0;
    else if (p->llink == NULL && p->rlink == NULL)
        return 1;
    else
        return leavesCount(p->llink) + leavesCount(p->rlink);
}

```

25. Add the operation queue Count to the class queue (the array implementation of queues), which returns the number of elements in the queue.

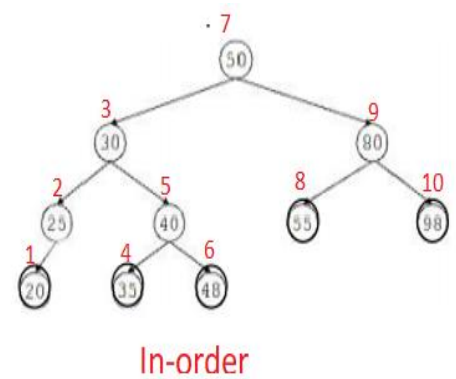
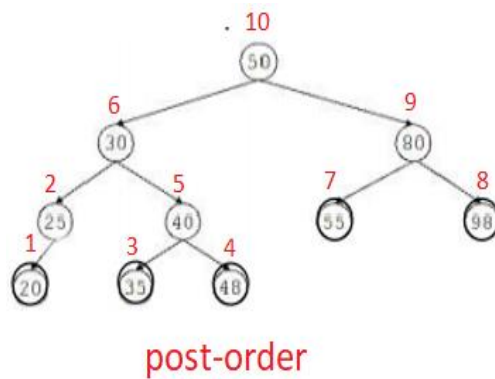
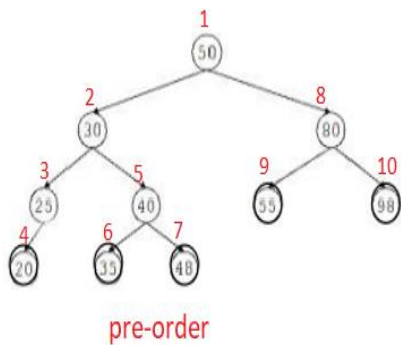
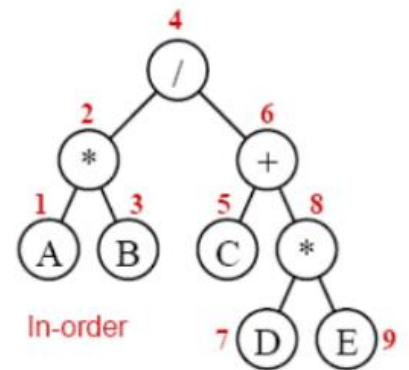
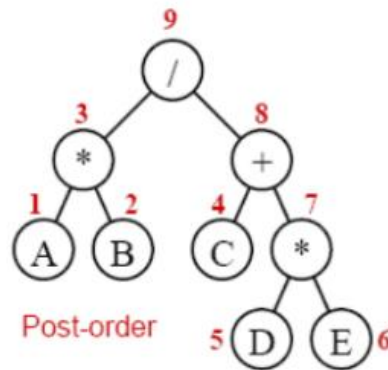
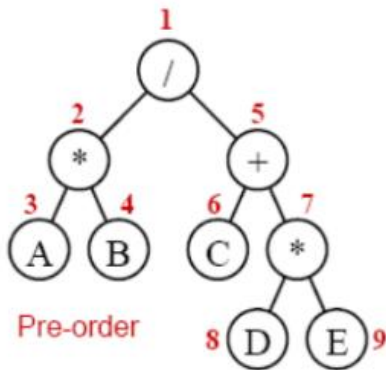
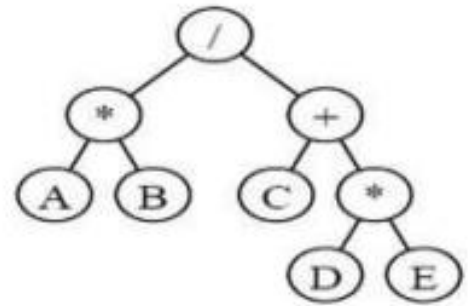
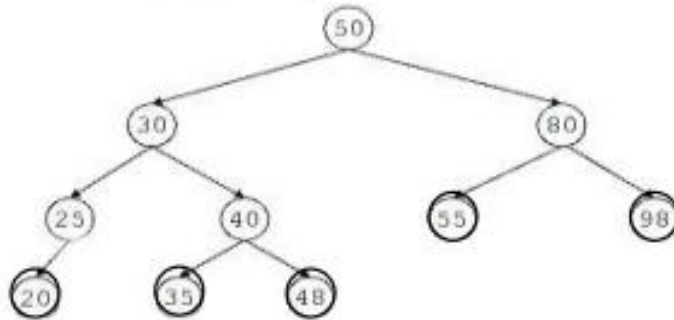
```
int getSize () {  
    int siz=0;  
    if (isEmpty()) {  
        cout << "Queue is Empty" << endl;  
        return 0;  
    } else {  
        for ( int i = Front; i <= rear; i++) {  
            siz++;  
        }  
        return siz;  
    }  
}
```

26. List the nodes of the following binary tree in:

1. preorder sequence.

2. postorder sequence

3. Inorder sequence



27. Translate the following infix expressions into prefix and then evaluate it:

1. $(6 * 4) / (7 - (3 + 2)) = 12$

Character	Stack	Ouput
+)	)	
+)	))	
+ 2	))	2
+ +	)) +	2
+ 3	)) +	2 3
+ (	)	2 3 +
+ -	) -	2 3 +
+ 7	) -	2 3 + 7
+ (		2 3 + 7 -
+ /	/	2 3 + 7 -
+)	/)	2 3 + 7 -
+ 4	/)	2 3 + 7 - 4
+ *	/) *	2 3 + 7 - 4
+ 6	/) *	2 3 + 7 - 4 6
+ (	/	2 3 + 7 - 4 6 *

Reverse final output.	
Prefix:	/ * 6 4 - 7 + 3 2

2. $(2 + 7) * (8 / (6 - (3 + 1))) = 36$

Character	Stack	Ouput
$\oplus$)	)	
$\oplus$)	))	
$\oplus$)	)))	
$\oplus$ 1	)))	1
$\oplus$ +	))) +	1
$\oplus$ 3	))) +	13
$\oplus$ (	))	13 +
$\oplus$ -	)) -	13 +
$\oplus$ 6	)) -	13 + 6
$\oplus$ (	)	13 + 6 -
$\oplus$ /	) /	13 + 6 -
$\oplus$ 8	) /	13 + 6 - 8
$\oplus$ (		13 + 6 - 8 /
$\oplus$ *	*	13 + 6 - 8 /
$\oplus$)	*)	13 + 6 - 8 /
$\oplus$ 7	*)	13 + 6 - 8 / 7
$\oplus$ +	*) +	13 + 6 - 8 / 7
$\oplus$ 2	*) +	13 + 6 - 8 / 7 2
$\oplus$ (	*	13 + 6 - 8 / 7 2 +
Reverse final output.		
Prefix:	* + 2 7 / 8 - 6 + 3 1	

28. Evaluate the following postfix expressions:

1. $35\ 14\ 4\ 5\ 15\ 3\ /\ * - - /\ 16 + = 17$

→ PostFix to inFix:-


$$\begin{aligned} &35\ 14\ 4\ 5\ 15\ 3\ /\ * - - /\ 16 + \\ &35\ 14\ 4\ 5\ (15/3)\ * - - /\ 16 + \\ &35\ 14\ 4\ (5*(15/3))\ - - /\ 16 + \\ &35\ 14\ (4-(5*(15/3)))\ - /\ 16 + \\ &35\ (14-(4-(5*(15/3))))\ /\ 16 + \\ &(35/(14-(4-(5*(15/3)))))\ /\ 16 + \\ &((35/(14-(4-(5*(15/3))))) + 16) = 17 \end{aligned}$$

#

2. $12\ 25\ 5\ 1\ /\ /\ * 8\ 7 + - = 45$

PostFix to inFix:-

$$\begin{aligned} &12\ 25\ 5\ 1\ /\ /\ * 8\ 7 + - \\ &12\ 25\ (5/1)\ /\ * 8\ 7 + - \\ &12\ (25/(5/1))\ * 8\ 7 + - \\ &(12 * (25/(5/1)))\ 8\ 7 + - \\ &((12 * (25/(5/1)))\ (8+7))\ - \\ &((12 * (25/(5/1))) - (8+7)) = 45 \end{aligned}$$

29. Translate the following infix to post fix and prefix using Stack:

1. $A * ((B + C) * (E - F) - G) * (H - I)$

29 (inFix $\rightarrow$ PreFix)



$$A * ((B + C) * (E - F) - G) * (H - I)$$

$$A * (+ BC * - EF - G) * - HI$$

$$A * (* + BC - EF - G) * - HI$$

$$A * - * + BC - EFG * - HI$$

$$** A - * + BC - EFG - HI$$

#

~~29~~

29 InFix $\rightarrow$ PostFix



$$A * ((B + C) * (E - F) - G) * (H - I)$$

$$A * (BC + * EF - - G) * HI -$$

$$A * (BC + EF - * - G) * HI -$$

$$A * BC + EF - * G - * HI -$$

$$ABC + EF - * G - ** HI -$$

$$ABC + EF - * G - * HI - *$$

#

30. Show what output by the following segment of code is:

```
stackarray stack;  
  
int x=4,y=10;  
  
stack.push(7);  
  
stack.push(x + 5);  
  
y = stack.top( );  
  
stack.pop();  
  
stack.push(x + y);  
  
x = stack.top();  
  
stack.pop();  
  
cout << "x = " << x << endl;  
  
cout << "y = " << y << endl;  
  
while (!stack.isEmptyStack())  
{cout << stack.top() << endl;  
stack.pop();  
}
```

Output:

X = 13

Y = 9

7

31. What is the output of the following program segment?

```
linkedStackType<int>myStack;  
myStack.push(10);  
myStack.push(20);  
myStack.pop();  
cout<<myStack.top()<<endl;  
myStack.push(25);  
myStack.push(2*myStack.top());  
myStack.push(-60);  
myStack.pop();  
while( myStack.isEmptyStack())  
{ cout<<tempStack.top()<<"";  
tempStack.pop(); }  
cout<<endl;  
cout<<myStack.top()<<endl;
```

output:

10

50